Open-ViT-bench: extending C-ViT in GPU

"Emanuele Poiana": Mat: 247176, emanuele.poiana@studenti.unitn.it, *GitRepo:*
https://github.com/IlPoiana/Open-VIT-bench.git

***Abstract*—**

I. Introduction

Vision Transformer (ViT) was a revolutionary adaptation from text sequence to image processing of the transformer model back in 2020 [1]. The core idea was to treat each image as a sequence of patches to leverage the multi-head attention (MHA) mechanism incorporated in the encoder and decoder block, proven superior by previous studies when compared to sequence models like RNN.

It has been successful and widely adopted, improved and optimized over the recent years and is still found as a backbone or reference for many modern applications in computer vision.

The architecture is designed for parallel processing and defines a series of operations, notably the MHA, with high Arithmetic Intensity (AI). This opens to several approaches for optimizing and accelerating the model especially on GPUs.

II. Problem Statement

I focused on implementing the original ViT model based on the C-ViT implementation using only SIMD kernels. My work had three main objectives: first, to build the model while balancing the tradeoff between hand-crafted kernels, offering greater flexibility but requiring more development time and library-based components, providing high performance but limiting customization options. Second, to test and validate the correctness of the model implementation. Third, to accelerate ViT pipeline once constructed, by optimizing data flow between components and looking for potential future improvements.

III. State of the Art

The landscape of deep learning acceleration has evolved across hardware, architectures, and software tooling. On the hardware front, GPUs have been revolutionized with the introduction of Tensor Cores, which offers specialized hardware for matrix operations. Distributed training frameworks have further enabled model scaling across multiple GPUs. Then on the Vision Transformer architecture, it has seen a remarkable evolution since its introduction. Variants like TinyViT [2] and FastViT [3] have optimized for resource constraints and inference speed, while techniques such as KV caching [4] and Multi-Head-Latent Attention [5] have addressed memory constraints.

Newer software tools developed alongside these advances. The CUDA Development Toolkit, in particular cuBLAS and cuDNN recent versions provide highly optimized deep learning primitives [6].

IV. Methodology and Contributions

My contributions for this project are as follows.

1) **Algorithm and library selection:** Identification of ViT fundamental operations, evaluation of available libraries and their capabilities and implementation of the key routines with them where possible.
2) **Architectural design:** Construction of a code structure based on the theoretical model and C-ViT, aimed at understanding the execution flow and how operations are managed in practice.
3) **Model implementation:** Development of the model and testing its correctness.
4) **Performance analysis:** Evaluation and comparison of the designed optimization techniques applied to the architecture and its components.
5) **Discussion of alternatives:** Assessment of unexplored approaches and potential refinements for future work.

A. Library choices

I focused on NVIDIA's state-of-the-art libraries available on the cluster for expensive operations including MLP, MHA, convolutional layers, layer normalization, and softmax.

cuDNN in particular is theoretically the optimal tool for this task, though the version I used was poorly documented and challenging to work with. I successfully implemented MHA using cuDNN's fused kernel. However, I was unable to implement layer normalization through cuDNN, so I developed different kernels enhancing CUB(see later section). I also implemented the convolutional layer(conv2d) and softmax using cuDNN. Softmax and conv2d was definetly easier to implement than MHA, particularly after gaining experience with cuDNN's infrastructure.

For MLP/GEMM, cuDNN's fused engines was even harder than mha in version 8.9.2 so I switched to cuBLASLt, which, unlike cuBLAS, accepts device pointers directly as inputs and outputs while maintaining asynchronous host-device behavior.

B. Layer Norm

Initially I tried to use cuDNN for this operation, but I was not able to set the available layer norm implementation for my version as the ViT, so I decided to implement mine.

The core of this operation kernel design, is that all computations are local only to the single token embeddings. The kernel assigned one or more tokens to process for each block, and each thread process one or more elements of the 768 embeddings, equally distributed among the threads block. For the reduction I initially hand designed a parallel reduction algorithm, to later discover CUB library implementation was

tailored for my case. Sequentially I integrate CUB parallel reduction necessary for mean and variance computation. The bias and scale vectors are the same for each token and so image, so I can also load once every thread specific bias and scale used for their processed token embeddings. This scales well with the multi-token strategy adopted.

Finally I can set macros for the embeddings size and tokens for each block, opening for loop unrolling.

C. Mlp

The Multi-Layer Perceptron, together with the linear layer in the prediction head, is responsible for the most expensive operations in ViT alongside MHA. The MLP structure is simple: one hidden layer with an intermediate GELU activation function.

The GEMMs are performed through cuBLASLt due to its ability to fuse bias or GELU epilogues into the kernel. However, the library version used doesn't support both epilogues simultaneously. To work around this, I treat the bias as the C tensor in the $D = \alpha(A * B) + \beta C$, which requires preprocessing the bias to match the linear output size. Additionally, the resulting d_t tensor, tensor must be transposed because cuBLASLt outputs D in column-major format while the system is designed in row-major.1

I also propose an alternative version using a separate fused kernel for bias and GELU instead of the fused epilogue. This approach avoids the increased memory usage for the bias and eliminates the final transpose required by the fused approach.

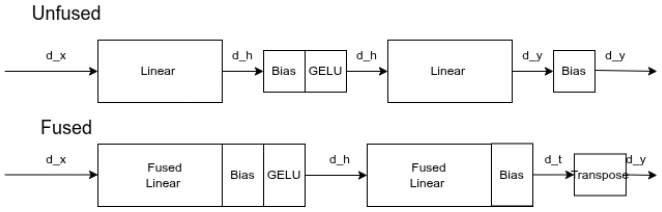

Fig. 1. Fused vs Unfused kernel implementations: on top the unfused does 4 separates kernels while the fused one just 2

D. Convolutional layer, Multi-head attention and Softmax

For these three important components I decided to use cuDNN. For both mha and conv2d operations I have to set flags to ensure no tensor core usage. While for the Softmax the process was straightforward, for the conv2d and especially the mha many host preparations are required(create, set and destroy descriptors). The mha in particular requires a custom weights load other than the cuDNN descriptors. The mha offers also the residual fusion, but for this case there have to be the possibility to scale the output of the mha before the residual add, so I avoided it and build a different simple kernel to do so.

E. Patch embedder

This implementation is similar to the one of C-ViT with two exceptions. Integrating cuDNN conv2d requires a custom transpose op. which results in the batch linearization obtaining $[B, T, E]$ tensor. I handle the CLS token add to each sequence as a on device memory copy of the data on a $[B, T + 1, E]$ tensor.(See Fig. 2) The H2D memory copy is made asynchronously.

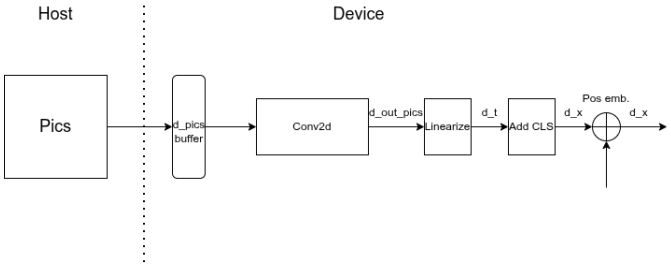

Fig. 2. Patch embedder implementation: each rectangle represent a kernel op. while d_pics is the buffer where the picture to process are initially set on the device memory

F. Encoder block

Each operations represented by the Figure 3 is present with the same logic in my system version. The only difference is the scale factor, which in the mha, is not possible to exploit the residual connection fusion.

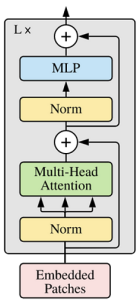

Fig. 3. Encoder block implementation: original ViT encoder block

G. Prediction head

The prediction head is also a composition of basic operations, where I chose to put the final argmax for the class prediction on the host side because it's a low AI operation. (See Fig. 4)

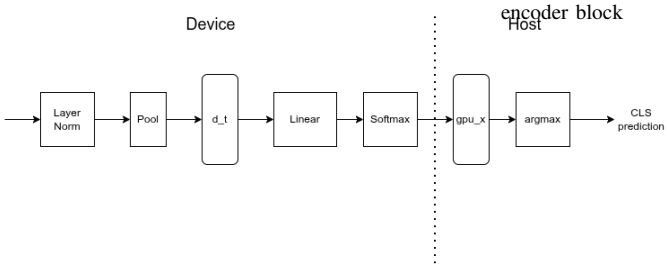

Fig. 4. Prediction head implementation: divided into device and host part. Starts with a $[B, T, E]$ tensor, then reduced to d_t $[B, cls]$ tensor and finally to $[B]$ array containing the predicted classes

H. ViT

Once every component is made, ViT can be made by the composition of patch embedder, a series of encoder blocks and a prediction head(see Fig. 7). One important aspect is memory capacity of the GPU, which determines the maximum batch size. Scaling the

application requires a form of looping from the host to load batches of data(pictures in this case) on the device and process them, also the final host computation of the prediction head represent a synchronization barrier.

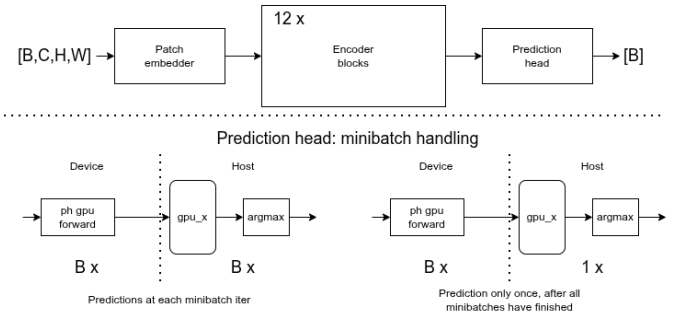

Fig. 5. **GPU-ViT forward schema**

There isn't a single way to achieve this, I propose three different approaches to this problem:

1) **Direct**: process the maximum batch size in the model depending on the available hardware and loop through each batch. This saves host memory but requires B host predictions which generates $B-1$ barriers between the device and the host
2) **One-prediction** : process the maximum batch size in the model like the "direct" approach, but accumulate asynchronously on the host side, where memory is tipically larger in size, all the probabilities vectors and compute each image class in one shot. This eliminate the host barrier between different batches forward.
3) **Multi-stream**: using one-prediction, instantiate different GPU-ViT instances, assigning to each of them a different CUDA stream. They can share the weights in inference and can process proportionally to the nuber of streams, smaller batches individually but the same dimension of the precedent methods together. This effectively tiles the input in many smaller minibatches, which are faster to load and the parallel ViT instances computes in one from each other without barriers.(except the final host prediction)

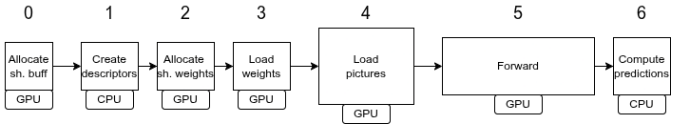

Fig. 6. **GPU-ViT operations execution order**: Operation from 0 to 3 are performed only at the beginning

V. System Description and Experimental Set-up

A. System Description

All the experiments conducted, were performed on the Unitn "baldo" cluster. Due to the long waiting time for obtaining some specific GPU, some test have been made with the NVIDIA A30 while others with the NVIDIA L40S. Is always

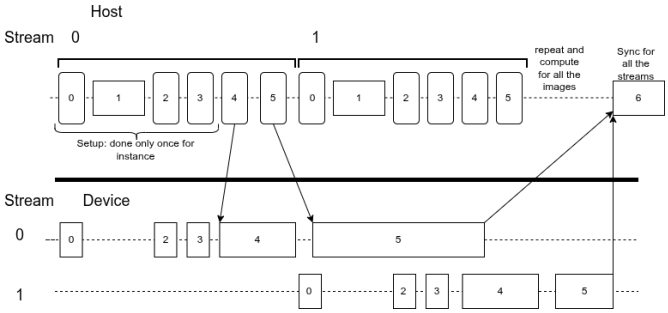

Fig. 7. **GPU-ViT multi-stream schema**: computations blocks reference from 6

referenced what GPU have been used to compute certain data or results. More info on the GPU technical details on VIII For this project, only the SIMD FP32 have been used.

B. Experimental Set-up

For components, I set 20 warm up runs and 100 test runs to average, for ViT 5 and 10. I've adopted MRE for the CPU/GPU difference with a minimum normalization factor of 1e-3 because using half fp16 storage. Each numerical test and benchmark have been tested on randomly generated data. MHA workspace size for single model instance 8GB, multi stream 512MB, this as a good balance for workspace, weights and images buffer sizes. I've not actively searched for optimal parameters because the focus of this project is is to show the optimizations done and how effect the model performance and scalability, not to obtain the fastest model possible through a parameter grid search.

VI. Experimental Results

For the components memory bounded I compute the effective bandwidth(GB/s) while for the compute bounded I compute the effective computational throughput (TFLOPs/s). I have done this calculation only for the single components.

A. Components efficiency

1) Layer norm: The layer norm achieved high effective bandwidth thanks to CUB introduction, which handles optimally the parallel reduction also at warp level. Then the multiple token and elements approach maximize the threads occupancy if opportunely tuned. Loop unrolling reduced instruction count and so total kernel operations. All kernel comparison tests have been made with batch size B 512 in an NVIDIA A30.

Kernel	T/B	E/T	Time (ms)	BW %
Naive	1	2	2.56	13%
CUB	1	2	1.04	32%
CUB-mt	4	2	0.91	36%
CUB-mte	4	4	0.54	61%
CUB-mte-unr.	4	4	0.48	69%

TABLE I
Layer Norm performance (T/B: tokens/block, E/T: elements/thread, BW: bandwidth efficiency). A30GPU

2) Softmax: It doesn't achieve a good effective bandwidth. I don't have control over it at least this version. So possible improvements here are kernel fusion with the linear layer and looking for other implementations.

3) Multi layer perceptron: The mlp has not achieved high computational throughput as expected. This can be a bad cuBLASLt configuration or that both proposed kernels have negative sides that nullify the optimizations. For example unfused do less FLOPs but launches more kernels, while the fused have to load an entire $[B, T, E]$ tensor for the bias. Also the cost for the kernel fusion in this case isn't really superior to the unfused method if not equal. This might be to the majority of computation time spend by the kernel doing GEMM operations not epilogues.

4) Conv2d: The convolutional layer hasn't reached high computational throughput as expected. This can possibly be a bad configuration on cuDNN.

5) Multi head attention: The mha have reached an acceptable computational throughput, in the operations of the mha appears the softmax which has a low AI and maybe stops the kernel to reach really high efficency.

All the reported results of this table habe been tested with batch size of B 2048 on a NVIDIA L40S.

TABLE II
PERFORMANCE ANALYSIS OF VIT OPERATIONS. NVIDIA L40S

Operation	Time (ms)	Metric	Peak %
Memory-Bound Operations			
Layer Norm	1.81	684.7 GB/s	79.8%
Softmax	0.0038	217.9 GB/s	25.2%
Compute-Bound Operations			
MLP	0.118	32.26 TFLOPS	35.2%
Conv2d	0.024	20.6 TFLOPS	22.5%
MHA	0.037	57.2 TFLOPS	62.4%

B. Patch embedder, encoder block and prediction head

The params settings for these components was irrelevant, due to the tiny contribute the add, pos. embeddings, scale and transpose. To process large datasets, the **patch embedder** might load the pictures from a pinned host buffer for faster h2d and d2h data transfer, choosing carefully the host buffer size to not get performance degradation. For the **prediction head**, An improvement is to switch the layer norm after the pool. The pool op. in this architecture keeps only the CLS token, passing from a 3D to a 2D tensor, so the layer norm, which acts only on a token level, can be safely switched and reducing by a factor equal to the sequence length the layer norm computation.

C. ViT

The single prediction approach doesn't have a strong impact respect the direct model, this probably to the fast computation of the CPU which doesn't produce a noticeable difference with this input sizes. The multi stream approach shows significant improvements, already with two streams. This comes from the asynchronous load of the pictures actually pipelining the different instances and from the better GPU usage, suggested by the low effective bandwidth and compute of the main kernels.

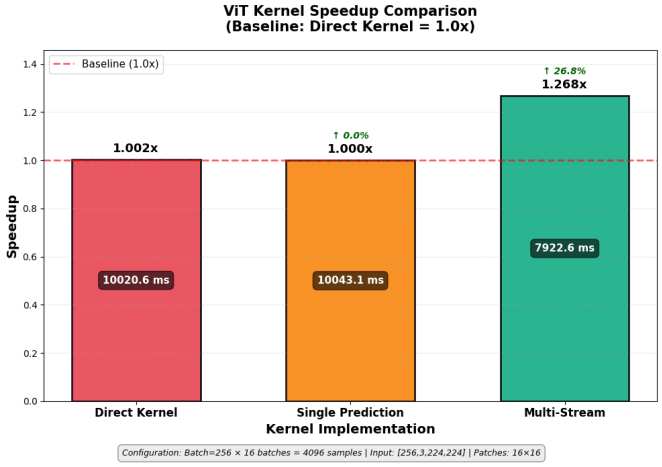

Fig. 8. **Vit kernel comparison**: Executed on NVIDIA L40S. stream number for multi stream: 2 (2 x 128 minibatches)

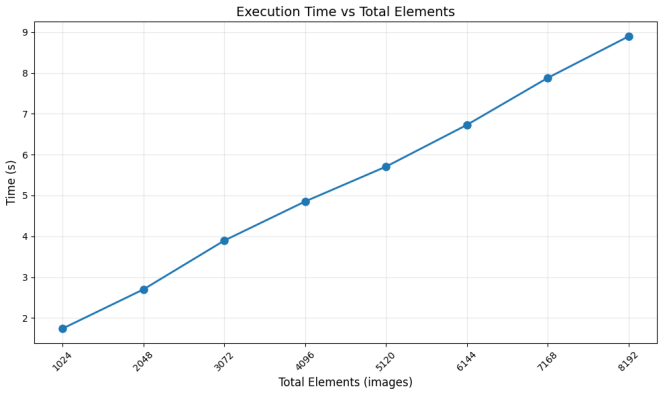

Fig. 9. **Vit multi-stream kernel scaling**: Executed on NVIDIA L40S. Streams 16, batch number 16

VII. CONCLUSIONS

Overall the model works and scales well, there are many promising areas of improvement. More kernel fusion for sure reduce execution times and increases AI for low AI kernels, enableing tensor cores also will boost the achievable TFLOPs. While modifying the architecture with modern solutions, like KVcaching can reduce the memory footprint of the model for a better scaling.

REFERENCES

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: https://arxiv.org/abs/2010.11929

[2] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu, and L. Yuan, "Tinyvit: Fast pretraining distillation for small vision transformers," 2022. [Online]. Available: https://arxiv.org/abs/2207.10666
[3] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, "Fastvit: A fast hybrid vision transformer using structural reparameterization," 2023. [Online]. Available: https://arxiv.org/abs/2303.14189
[4] H. Wu and K. Tu, "Layer-condensed kv cache for efficient inference of large language models," 2024. [Online]. Available: https://arxiv.org/abs/2405.10637
[5] DeepSeek-AI, A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Yang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Chen, J. Yuan, J. Qiu, J. Song, K. Dong, K. Gao, K. Guan, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Pan, R. Xu, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Zheng, T. Wang, T. Pei, T. Yuan, T. Sun, W. L. Xiao, W. Zeng, W. An, W. Liu, W. Liang, W. Gao, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Chen, X. Nie, X. Sun, X. Wang, X. Liu, X. Xie, X. Yu, X. Song, X. Zhou, X. Yang, X. Lu, X. Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Zheng, Y. Zhang, Y. Xiong, Y. Zhao, Y. He, Y. Tang, Y. Piao, Y. Dong, Y. Tan, Y. Liu, Y. Wang, Y. Guo, Y. Zhu, Y. Wang, Y. Zou, Y. Zha, Y. Ma, Y. Yan, Y. You, Y. Liu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Huang, Z. Zhang, Z. Xie, Z. Hao, Z. Shao, Z. Wen, Z. Xu, Z. Zhang, Z. Li, Z. Wang, Z. Gu, Z. Li, and Z. Xie, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024. [Online]. Available: https://arxiv.org/abs/2405.04434
[6] NVIDIA, "cudnn 8.9.2." [Online]. Available: https://docs.nvidia.com/deeplearning/cudnn/archives/cudnn-892/api/index.html#cudnnMultiHeadAttnForward

VIII. Additional Material

A. System specs table

TABLE III
System Specifications: A30 on the top, L40S on the bottom

Specification	Value
Memory Bandwidth	933 GB/s
FP32 Performance	10.3 TFLOPs
Specification	**Value**
Memory Bandwidth	864 GB/s
FP32 Performance	91.6 TFLOPs

TABLE IV
Software specifications

Software	version
CUDA	12.1.1
cuDNN	8.9.2
python	3.10.12

Processor	Model	Cores	RAM
CPU	AMD EPYC 9334	2x32 at 2.7 GHz	251GB
GPU	Nvidia A30	3584 FP32 CUDA Cores at maximum 1.44 GHz	24GB HBM2
GPU	Nvidia L40S	18,176 FP32 Ada-Lovelace CUDA Cores	48GB GDDR6

TABLE V
System details

B. Layer norm pseudocode

Algorithm 1 Multi-Element CUB Layer Norm

procedure DevLN($idx, g\_idx, in, out, s, b, \epsilon$)
  shared μ, σ^2
  Alloc $data[E]$, $buff[E]$
  for $i = 0$ to $E - 1$ **do** ▷ Load
    $data[i] \leftarrow in[g\_idx + B \cdot i]$
    $buff[i] \leftarrow data[i]$
  end for
  $agg \leftarrow$ BlkReduce.Sum($data$)
  if $idx = 0$ **then** $\mu \leftarrow agg/c$
  end if
  `__syncthreads()`
  for $i = 0$ to $E - 1$ **do** ▷ Variance
    $data[i] \leftarrow (buff[i] - \mu)^2$
  end for
  $agg \leftarrow$ BlkReduce.Sum($data$)
  if $idx = 0$ **then** $\sigma^2 \leftarrow agg/c$
  end if
  `__syncthreads()`
  for $i = 0$ to $E - 1$ **do** ▷ Normalize
    $o\_idx \leftarrow g\_idx + B \cdot i$
    `ln`$(in, out, \mu, \sigma^2, b, s, \epsilon, o\_idx, i)$
  end for
end procedure

Algorithm 2 Layer Norm Kernel Loop

procedure LNKernel($x, y, scale, bias, \epsilon$)
  $l\_idx \leftarrow threadIdx.x$
  $g\_idx \leftarrow blockIdx.x \cdot blockDim.x + l\_idx$
  for $i = 0$ to $E - 1$ **do** ▷ Load scale/bias
    $th\_b[i] \leftarrow bias[l\_idx + i \cdot D]$
    $th\_s[i] \leftarrow scale[l\_idx + i \cdot D]$
  end for
  for $t = 0$ to $T - 1$ **do** ▷ Process tokens
    $idx \leftarrow g\_idx + S \cdot t$
    DevLN($l\_idx, idx, x, y, th\_s, th\_b, \epsilon$)
  end for
end procedure